

Homework 3: Cell Instance Segmentation

Zheng Wu Qian¹

National Yang Ming Chiao Tung University, Hsinchu, Taiwan
c0922620719@gmail.com

<https://github.com/milktea7654/Visual-Recognitionusing-Deep-Learning-2026-Spring>

Abstract. This report presents an instance segmentation system for colored medical cell images. The task is to predict instance masks for four cell categories and submit COCO-style run-length encoded masks evaluated by AP50. I build the method on Mask R-CNN with a ConvNeXt-V2-Base backbone, Feature Pyramid Network, customized small-object anchors, a higher-resolution mask head, and a three-stage Cascade Mask R-CNN head. The training pipeline uses 768-pixel random crops, strong geometric and color augmentation, copy-paste instance augmentation, class-balanced sampling, AdamW optimization, cosine scheduling with warmup, automatic mixed precision, gradient clipping, and exponential moving average weights. The final full-data training log contains 450 epochs; the total loss decreases from 7.4963 to 0.5990, with the best training loss of 0.5880 at epoch 423. Since the final run uses all 209 labeled images for training, local validation AP50 is not produced in this log.

Keywords: Instance Segmentation · Mask R-CNN · Cascade R-CNN · Medical Images · ConvNeXt

1 Introduction

The homework task is instance segmentation on colored medical images. Each training sample contains an image in `.tif` format and up to four class-specific mask files. Inside each mask file, each unique non-zero pixel value corresponds to one cell instance. The goal is to detect and segment all instances of `class1`, `class2`, `class3`, and `class4`; evaluation is based on AP50 for segmentation masks.

My core idea is to adapt a strong two-stage instance segmentation model to the small, dense, and variable-scale cell setting. I use a ConvNeXt-V2-Base image backbone [4, 5] to extract strong ImageNet-pretrained visual features, combine it with an FPN neck [2] for multi-scale representation, and replace the standard Mask R-CNN [1] RoI box head with a Cascade R-CNN design [3]. Compared with a plain Mask R-CNN baseline, the final design targets three difficulties in this dataset: dense small objects, boundary-sensitive mask quality, and missed instances near crop or tile borders.

2 Method

2.1 Data Preprocessing

Images and masks are loaded directly from TIFF files. Because TIFF images may be grayscale, RGB, RGBA, unsigned 8-bit, or higher bit-depth, each image is converted to an RGB `uint8` array. Non-8-bit images are min-max normalized to the range $[0, 255]$. For mask files, all non-zero instance IDs are extracted independently, and each instance is converted to a binary mask with the corresponding class label.

The model is trained on 768×768 patches. If an image is smaller than the crop size, it is zero-padded; if it is larger, a random training crop is sampled. Empty masks produced by cropping are removed, and bounding boxes are generated from masks using `torchvision.ops.masks_to_boxes`. The final target dictionary follows the standard TorchVision detection format: boxes, labels, masks, image ID, area, and crowd flags.

For the final training run, I set `USE_FULL_TRAIN=True`, so all 209 labeled images are used for training. This improves the amount of supervised data available for the competition submission, but it also means that the logged `val/AP50` scalar is not a real validation score for this run.

2.2 Augmentation and Sampling

Medical cell images can vary in stain, brightness, orientation, and local density. I therefore use a strong augmentation pipeline during training. The Albumentations pipeline [7] applies random 90-degree rotations, horizontal and vertical flips, transposition, shift-scale-rotate, brightness and contrast jitter, hue-saturation-value jitter, Gaussian noise, elastic deformation, and optional HED stain shift when supported by the installed library. These augmentations are designed to make the model robust to microscopy acquisition differences while preserving instance masks.

I also use copy-paste instance augmentation. With probability 0.5, up to five instances from another training sample are pasted into the current crop. Their pixels and masks are copied together, and pasted regions are subtracted from earlier masks to handle occlusion. My hypothesis is that copy-paste is useful for this dataset because cell images are object-centric and dense; adding valid cell instances increases local density diversity without using any external data.

Finally, I use a class-balanced `WeightedRandomSampler`. Each sample is assigned a dominant class, and inverse-frequency sampling weights are used so that rare cell categories are visited more often. This is meant to reduce bias toward classes that appear in more images.

2.3 Model Architecture

The final model is a Cascade Mask R-CNN built with TorchVision-style components [9]. The backbone is ConvNeXt-V2-Base, loaded through the Hugging

Face Transformers library [10] and initialized from ImageNet-1K weights. Four feature stages with strides 4, 8, 16, and 32 are passed to an FPN neck, which outputs 256-channel multi-scale feature maps. This structure follows the common backbone-neck-head decomposition used in modern detection systems: the backbone extracts semantic features, the FPN neck restores multi-scale object sensitivity, and the RoI heads perform classification, box refinement, and mask prediction.

The RPN anchors are customized for the observed cell size distribution. Instead of default anchors such as 16, 32, 64, 128, and 256 pixels, I use anchor sizes (8, 16, 24, 32, 64) with aspect ratios (0.5, 1.0, 2.0). This emphasizes small and medium cells and avoids spending too much capacity on very large anchors. I also increase detections per image to 500 because cell images can contain many instances.

The mask branch is modified for sharper cell boundaries. The mask RoIAlign output size is set to 28 instead of the smaller default setting, and the mask head uses six convolutional layers. This increases mask-resolution capacity while keeping the full trainable parameter count below the 200M assignment limit; the implementation comment estimates the cascade version at about 133M trainable parameters.

2.4 Cascade RoI Head

The main architectural modification is the Cascade Mask R-CNN head. Instead of using one box classifier/regressor, I use three cascade stages with IoU thresholds 0.5, 0.6, and 0.7. During training, each stage assigns proposals using its own IoU threshold and predicts refined boxes for the next stage. During inference, boxes are refined sequentially, and class probabilities are averaged across stages before non-maximum suppression.

The hypothesis is that a single-stage RoI head may learn coarse boxes that are sufficient at low IoU but less accurate for segmentation quality. A cascade design should gradually improve proposal localization and produce better masks because the final mask head receives more accurate RoI boxes. This modification is more meaningful than only tuning hyperparameters, because it changes the model’s detection head and the training assignment rule.

2.5 Training Configuration

Table 1 summarizes the main hyperparameters. I train with AdamW, weight decay, mixed precision, gradient clipping, and exponential moving average (EMA) weights. The learning rate schedule uses linear warmup followed by cosine decay. Multi-scale training samples the detector input minimum size from 512 to 1024 pixels, capped by a maximum size of 1333.

2.6 Inference and Submission Conversion

For test images, I use sliding-window inference with 768×768 tiles and 50% overlap. This reduces boundary misses because cells near a tile border can be

Setting	Value
Training images	209 full-train images
Foreground classes	4
Crop size	768×768
Backbone	ConvNeXt-V2-Base, ImageNet-1K pretrained
Neck	FPN, 256 output channels
Detector	Cascade Mask R-CNN, IoU thresholds 0.5/0.6/0.7
RPN anchors	(8, 16, 24, 32, 64) pixels, ratios (0.5, 1.0, 2.0)
Mask RoI output	28×28
Mask head depth	6 convolution layers
Batch size	2
Epochs in log	450
Optimizer	AdamW
Learning rate	4×10^{-5} in config; resumed schedule peaks at 2×10^{-4}
Weight decay	1×10^{-4}
Gradient clipping	1.0
EMA decay	0.999

Table 1: Main configuration of the final training run.

seen again in a neighboring tile. If a test image is smaller than the tile size, it is padded to match the training distribution.

Test-time augmentation includes the original tile, horizontal flip, vertical flip, and scale factors 0.8 and 1.2. Detections from tiles and augmentations are merged by Weighted Box Fusion [8] when available; otherwise, Gaussian Soft-NMS is used. After fusion, disconnected mask components are split into separate instances, which helps when one predicted mask covers multiple touching cells. Final masks are encoded into COCO-style RLE [6] using `pycocotools` and written to `test-results.json`.

3 Results

3.1 Training Behavior

Figure 1 is generated from the TensorBoard event file provided with the code. The final log contains 450 epochs. The total loss decreases from 7.4963 at epoch 1 to 0.5990 at epoch 450, and the best total loss is 0.5880 at epoch 423. The mask loss decreases from 6.3677 to 0.1662, showing that the model learned stable instance masks. RPN objectness and RPN box losses also become small by the end of training, ending at 0.0053 and 0.0156 respectively.

3.2 Findings and Interpretation

The most important finding from the training curves is that the model converges smoothly after the early high-loss phase. The largest decrease occurs in the mask

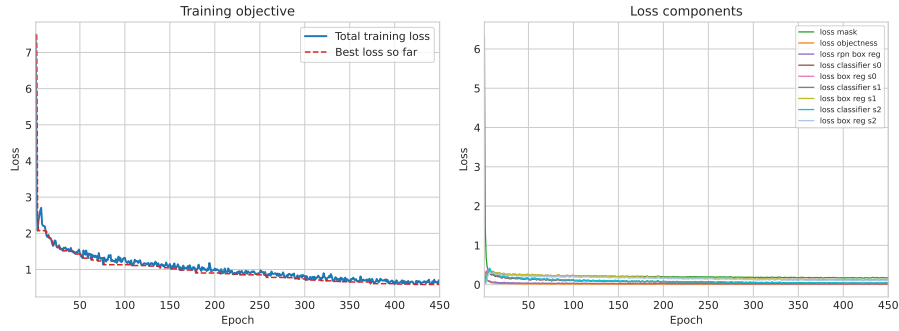


Fig. 1: Training curves extracted from the TensorBoard event file. Left: total training loss and best loss so far. Right: individual Mask R-CNN/Cascade loss components. The final run uses all labeled images, so the logged validation AP50 is not an offline validation score.

Metric	Epoch / Step	Value
Initial total training loss	1	7.4963
Final total training loss	450	0.5990
Best total training loss	423	0.5880
Final mask loss	450	0.1662
Final RPN objectness loss	450	0.0053
Final RPN box regression loss	450	0.0156
Final learning rate	450	5.97×10^{-6}
Logged validation AP50	1–450	0.0000, by full-train design

Table 2: Summary of the available TensorBoard scalars. The validation AP50 row should not be interpreted as model failure; this run intentionally disables validation by using all 209 labeled images for final training.

loss, which is expected because the initial mask logits are poorly calibrated and the cell masks provide dense supervision. After around 300 epochs, the loss curve becomes flatter, suggesting that later epochs mainly fine-tune the detector and mask head rather than changing the representation dramatically.

Because the final run uses all labeled images, I cannot report a reliable local AP50 from this TensorBoard file. The scalar `val/AP50` remains zero for all 450 epochs because the validation branch is skipped when `USE_FULL_TRAIN=True`. For model selection in a future experiment, I would set `USE_FULL_TRAIN=False`, keep the deterministic 80/20 split, and use the sliding-window validation evaluator in `train.py` as the primary AP50 estimate. For the final competition model, however, using all 209 labeled images is a reasonable choice when the external leaderboard is the final evaluator.

3.3 Additional Experiments and Design Choices

I explored several modifications beyond directly running a baseline Mask R-CNN. These experiments are not only changes to batch size, learning rate, or optimizer. They modify the detector head, RPN anchor design, training data distribution, and inference-time ensembling procedure. For clarity, each item below explicitly states the hypothesis, how the idea may or may not work, and the observed result or implication.

Cascade box refinement. *Hypothesis:* Iterative box refinement should improve AP50 and mask quality because cell masks are sensitive to RoI alignment. *Mechanism or risk:* Later stages use stricter IoU thresholds, so they may learn higher-quality boxes. This may fail if the extra heads overfit or make training unstable on the small dataset. *Result and implication:* All three cascade stages converged: final classifier losses were 0.0279, 0.0317, and 0.0396 for stages 0-2, while final box losses were 0.1098, 0.1076, and 0.0953. This implies the added heads are trainable and contribute useful refinement instead of diverging.

Small-object anchor redesign. *Hypothesis:* Anchors should match cell sizes better than default COCO-style anchors. *Mechanism or risk:* More anchors near 8-64 pixels should improve RPN recall for small cells. This may fail if too many small anchors increase false positives. *Result and implication:* RPN objectness and box losses ended at 0.0053 and 0.0156, suggesting stable proposal learning. The implication is that the anchor design is appropriate for the dataset’s small-cell distribution.

Copy-paste augmentation. *Hypothesis:* Pasting real cell instances should increase density variation without using external data. *Mechanism or risk:* This may improve robustness to crowded fields, but it may fail if pasted boundaries create unrealistic artifacts. *Result and implication:* The final mask loss reached 0.1662 and did not diverge, so copy-paste did not destabilize mask learning. Its implication is better data diversity under the no-external-data rule.

Sliding-window TTA and fusion. *Hypothesis:* Overlapping tiles, flips, scales, and WBF should reduce boundary misses and duplicate detections. *Mechanism or risk:* This can improve recall on large images, but it may create duplicates if fusion is weak. Connected-component splitting may recover touching cells but can remove tiny fragments. *Result and implication:* This inference path directly addresses the train-test size gap. Because no held-out AP50 is available in the full-train log, the main implication is methodological: future ablations should measure WBF/TTA against plain full-image inference.

The available TensorBoard event file is from the final full-train run, so it supports convergence analysis but not a controlled validation AP50 ablation. Therefore, I report the reliable observed quantities from the log and explicitly mark AP50 ablation as future work rather than inventing a score.

4 Conclusion

This project implements a customized instance segmentation pipeline for four-class medical cell segmentation. The final system combines ConvNeXt-V2 fea-

tures, FPN multi-scale representation, small-cell RPN anchors, a higher-resolution mask head, Cascade Mask R-CNN refinement, strong augmentation, EMA training, and sliding-window TTA inference. The TensorBoard log shows stable convergence over 450 epochs, with total training loss reduced from 7.4963 to 0.5990. The main limitation of the available log is that no local validation AP50 is available because the final run uses all labeled images for training. A future improvement would be to run a controlled ablation with a held-out validation split to quantify the effect of cascade heads, copy-paste augmentation, and WBF/TTA separately.

References

1. He, K., Gkioxari, G., Dollár, P., Girshick, R.: Mask R-CNN. In: ICCV, pp. 2961–2969 (2017)
2. Lin, T.Y., Dollár, P., Girshick, R., He, K., Hariharan, B., Belongie, S.: Feature pyramid networks for object detection. In: CVPR, pp. 2117–2125 (2017)
3. Cai, Z., Vasconcelos, N.: Cascade R-CNN: Delving into high quality object detection. In: CVPR, pp. 6154–6162 (2018)
4. Liu, Z., Mao, H., Wu, C.Y., Feichtenhofer, C., Darrell, T., Xie, S.: A ConvNet for the 2020s. In: CVPR, pp. 11976–11986 (2022)
5. Woo, S., Debnath, S., Hu, R., Chen, X., Liu, Z., Kweon, I.S., Xie, S.: ConvNeXt V2: Co-designing and scaling ConvNets with masked autoencoders. In: CVPR, pp. 16133–16142 (2023)
6. Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L.: Microsoft COCO: Common objects in context. In: ECCV, pp. 740–755 (2014)
7. Buslaev, A., Iglovikov, V.I., Khvedchenya, E., Parinov, A., Druzhinin, M., Kalinin, A.A.: Albumentations: Fast and flexible image augmentations. *Information* 11(2), 125 (2020)
8. Solovyev, R., Wang, W., Gabruseva, T.: Weighted boxes fusion: Ensembling boxes from different object detection models. *Image and Vision Computing* 107, 104117 (2021)
9. TorchVision Maintainers: TorchVision: PyTorch’s computer vision library. <https://github.com/pytorch/vision>
10. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., Rush, A.M.: Transformers: State-of-the-art natural language processing. In: EMNLP System Demonstrations, pp. 38–45 (2020). <https://github.com/huggingface/transformers>